

Facultad de Ingeniería

Escuela de Informática

Ingeniería Civil en Informática

**Modelado, Implementación y Análisis del problema: Hamiltonian Cycle
(dirigido)**

Curso: Optimización

Integrantes: Daniel Rossel- Eduardo Diaz – Jean Guerrero.

Fecha: 26 de abril 2026



Índice

1. Introducción
2. Definición del problema
3. Modelo Matemático
4. Restricciones
5. Complejidad y NP-Compleitud
6. Implementación
7. Análisis Experimental
 - 7.1 Fuerza Bruta — Comparación entre lenguajes
 - 7.2 Backtracking con Poda — Comparación entre lenguajes

- 7.3 Fuerza Bruta vs Backtracking con Poda

8. Conclusiones

9. Referencias

1. Introducción

El Ciclo Hamiltoniano en Grafos Dirigidos (Directed Hamiltonian Cycle, DHC) es uno de los 21 problemas NP-completos clásicos identificados por Richard Karp [1] en 1972. Dado un grafo dirigido $G = (V, A)$, el problema consiste en determinar si existe un ciclo que visite cada nodo exactamente una vez y regrese al nodo de partida.

Este problema tiene aplicaciones en: planificación de rutas, diseño de circuitos, bioinformática (secuenciación de ADN) y logística.

2. Definición del problema

Descripción del problema:

Dado un mapa de ciudades conectadas por carreteras de sentido único, ¿existe una ruta que parta de una ciudad, pase por todas las demás exactamente una vez, y regrese al punto de inicio?

Definición formal

- **Entrada:** Un grafo dirigido $G = (V, A)$ donde V es el conjunto de nodos y $A \subseteq V \times V$ es el conjunto de aristas dirigidas.
- **Salida:** Sí, si existe un ciclo hamiltoniano (permutación π de V tal que $(\pi(i), \pi(i+1)) \in A$ para todo i , y $(\pi(n), \pi(1)) \in A$). NO, en caso contrario.

Ejemplo ilustrativo

Para el grafo con nodos $\{0, 1, 2, 3\}$ y aristas $\{0 \rightarrow 1, 1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 0\}$:

Ciclo hamiltoniano: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$ Sí

No válido: $0 \rightarrow 1 \rightarrow 0 \rightarrow 2 \rightarrow 3$ NO (nodo 0 repetido)

Aplicaciones

- Planificación de rutas de transporte y logística
- Diseño de circuitos impresos (PCB routing)
- Ensamblaje de genomas en bioinformática
- Problemas de scheduling con dependencias cíclicas

3. Modelo Matemático

- Conjuntos y parámetros

- $G = (V, A)$: grafo dirigido con $|V| = n$ nodos
- V : conjunto de nodos, indexados $0, 1, \dots, n-1$
- A : conjunto de aristas dirigidas. $(i,j) \in A$ indica que existe arista de i hacia j

- Variable de decisión

Se define una variable binaria para cada arista del grafo:

$$x_{\{ij\}} \in \{0, 1\} \quad \text{para todo } (i,j) \in A$$

$$x_{\{ij\}} = 1 \rightarrow \text{la arista } (i \rightarrow j) \text{ pertenece al ciclo hamiltoniano}$$

$$x_{\{ij\}} = 0 \rightarrow \text{la arista } (i \rightarrow j) \text{ no pertenece al ciclo}$$

4. Restricciones

- Restricción 1 — Salida única por nodo:

$$\sum_{\{j:(i,j) \in A\}} x_{\{ij\}} = 1 \quad \forall i \in V$$

De cada nodo debe salir exactamente una arista del ciclo.

- Restricción 2 — Entrada única por nodo:

$$\sum_{\{i:(i,j) \in A\}} x_{\{ij\}} = 1 \quad \forall j \in V$$

A cada nodo debe llegar exactamente una arista del ciclo.

- Restricción 3 — Eliminación de subciclos (MTZ):

$$u_i - u_j + n \cdot x_{ij} \leq n-1 \quad \forall (i,j) \in A, i \neq 1, j \neq 1$$

$$u_1 = 1, \quad u_i \in \mathbb{R} \quad \forall i \in V$$

La variable auxiliar u_i representa el orden de visita del nodo i . Esta restricción, debida a Miller, Tucker y Zemlin (1960) [3], impide la formación de subciclos que satisfagan las restricciones 1 y 2 sin ser un único ciclo hamiltoniano completo.

Versión de decisión

La respuesta es Sí, solo si existe una asignación de variables x_{ij} que satisfaga todas las restricciones anteriores simultáneamente.

Función Objetivo

No aplica. Al ser un problema de decisión pura, el objetivo no es optimizar una métrica, sino determinar la existencia de una asignación de variables que satisfaga simultáneamente todas las restricciones del modelo.

5. Complejidad y NP-Complejidad

Pertenencia a NP

Un problema pertenece a NP si, dado un certificado (solución candidata), existe un algoritmo polinomial que la verifica [2]. Para DHC:

- Certificado: una secuencia de nodos $\pi = (v_1, v_2, \dots, v_n, v_1)$
- Verificación en $O(n)$: recorrer la secuencia comprobando que (i) tiene n nodos distintos, (ii) cada par consecutivo tiene arista dirigida en A , (iii) la arista de cierre $(v_n \rightarrow v_1)$ existe

Verificar toma tiempo lineal $O(n)$, luego $DHC \in NP$.

NP-Duro

Se demuestra que DHC es NP-completo mediante reducción polinomial desde el Hamiltonian Path Dirigido (HP), que es NP-completo por reducción desde 3-SAT (Karp, 1972) [1].

Construcción: Dado un grafo $G = (V, A)$ y un nodo s , construimos $G' = (V, A \cup \{(v,s) : v \in V, v \neq s\})$. Es decir, se agrega una arista de cada nodo de regreso a s .

Equivalencia: G tiene un Hamiltonian Path desde s si y solo si G' tiene un Directed Hamiltonian Cycle. La construcción toma $O(n)$, luego $HP \leq_p DHC$.

Como HP es NP-completo y $HP \leq_p DHC$, se concluye que DHC es NP-completo.

Por qué no existe algoritmo eficiente conocido

La fuerza bruta requiere explorar $O(n!)$ permutaciones. Con backtracking, el peor caso sigue siendo exponencial $O(n!)$. No se conoce ningún algoritmo polinomial para DHC , y si $P \neq NP$ (conjetura ampliamente aceptada), ninguno existirá.

6. Implementación

Se implementaron dos soluciones en los tres lenguajes requeridos. La representación del grafo es una matriz de adyacencia $n \times n$.

Solución base — Fuerza bruta $O(n!)$

Genera todas las permutaciones del conjunto de nodos (fijando el nodo 0 como inicio para evitar contar el mismo ciclo n veces) y verifica si cada permutación constituye un ciclo hamiltoniano.

Para cada permutación π de $\{1, \dots, n-1\}$:

 candidato = $[0] + \pi$

 si $es_ciclo(adj, candidato)$: retornar candidato

retornar None

Solución mejorada — Backtracking con poda

Construye el camino nodo a nodo. En cada paso solo avanza si existe una arista dirigida hacia el vecino candidato y el vecino no ha sido visitado. Si no hay vecinos válidos, retrocede (backtrack). Al completar n nodos, verifica la arista de cierre.

función $backtrack(camino, visitado, profundidad)$:

si profundidad == n:

retornar adj[camino[-1]][camino[0]] == 1

para cada vecino v no visitado con arista desde actual:

agregar v al camino, marcar visitado

si backtrack(...): retornar True

deshacer (backtrack)

retornar False

Lenguajes y decisiones de implementación

La siguiente tabla muestra la estructura principal de las implementaciones en cada lenguaje y las observaciones. En la tabla1.

Lenguaje	Estructura principal	Observación
Python	list para camino, permutations de itertools	Más legible
C++	vector<int>, next_permutation de STL	Más rápido, compilado con -O2
Java	ArrayList<Integer>, permutación manual	Intermedio, JVM con JIT

Tabla1. Comparativa de estructura y principales diferencias

7. Análisis Experimental

Se midieron tiempos de ejecución en milisegundos para grafos dirigido: el ciclo hamiltoniano corresponde a la última permutación lexicográfica, forzando a la fuerza bruta a evaluar todas las permutaciones antes de encontrar la solución. Esto simula el peor caso real. Los tres lenguajes utilizan exactamente el mismo grafo (misma estructura, mismas aristas), permitiendo una comparación directa entre los 3.

7.1 Fuerza Bruta — Comparación entre lenguajes

La fuerza bruta evalúa todas las $(n-1)!$ permutaciones posibles. El número de permutaciones evaluadas fue el mismo en los tres lenguajes, lo que confirma que los algoritmos son equivalentes y solo difiere la velocidad de ejecución, como se puede observar en la siguiente tabla. En la tabla2.

n	permutaciones exploradas	Python (ms)	Java (ms)	C++ (ms)
8	5.040	2,3936	0,9469	0,2816
9	40.320	16,6482	1,7783	2,1335
10	362.880	149,5587	15,7758	17,691
11	3.628.800	1.482,49	49,4242	184,0204
12	39.916.800	16.574,04	366,4325	1.901,29
13	479.001.600	206.264,10	4.140,65	22.653,93

Tabla2. Comparativa de resultados de Fuerza Bruta entre Lenguajes.

En el siguiente gráfico realizado en Excel, se puede apreciar de mejor forma la comparación. En la Figura 1.

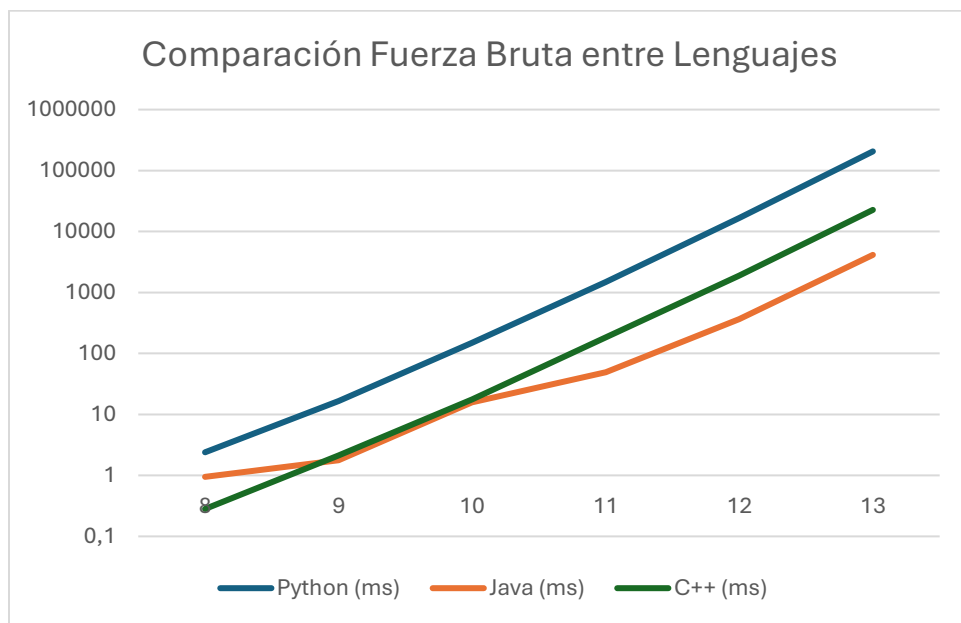


Figura 1. Tiempo de ejecución de la fuerza bruta por lenguaje. Escala logarítmica en eje Y.

Análisis

Python es consistentemente el más lento, a veces entre 8 y 9 veces más lento que C++. Esta diferencia se mantiene en todos los valores de n , lo que refleja el costo fijo del intérprete por operación. Por otro lado, se puede observar como Java iguala y supera a C++ a partir de $n=9$ y $n=10$, donde: el compilador JIT de la JVM detecta el bucle iterativo como código caliente tras ejecutarlo en $n=8$ y lo compila a código nativo, reduciendo su tiempo al nivel de C++ e incluso superándolo para esos valores.

7.2 Backtracking con Poda — Comparación entre lenguajes

El backtracking exploró exactamente el mismo número de nodos en los tres lenguajes. Las expansiones crecen de forma aproximadamente lineal (11 por cada n adicional), en contraste con el crecimiento factorial de la fuerza bruta como se puede ver en la siguiente tabla. En la tabla3.

n	expansiones	Python (ms)	Java (ms)	C++ (ms)
8	31	0,0192	0,0286	0,0019
9	40	0,0228	0,0212	0,0016
10	50	0,0299	0,0237	0,0016
11	61	0,0323	0,0345	0,0019
12	73	0,0392	0,0406	0,0022
13	86	0,0464	0,0508	0,0021

Tabla3. Comparativa de resultados de Backtracking con poda entre Lenguajes.

En el siguiente gráfico realizado en Excel, se pueden ver mejor las diferencias. En la Figura 2

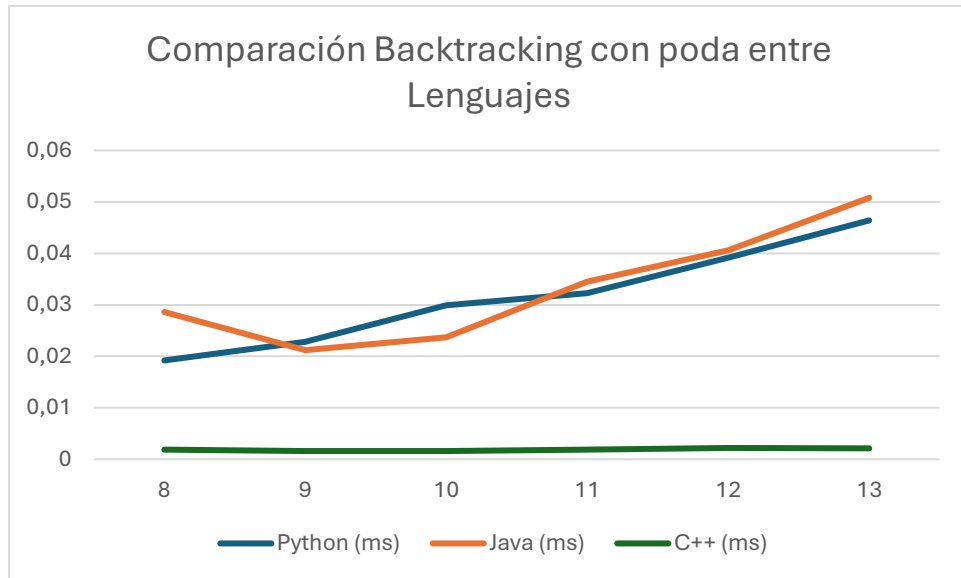


Figura 2. Tiempo de ejecución del backtracking con poda por lenguaje. Escala lineal.

En la siguiente tabla se aplica la misma implementación, pero hasta $n = 20$. En la tabla4.

n	Expansiones	Python (ms)	Java (ms)	C++ (ms)
8	31	0,031	0,0392	0,0016
9	40	0,027	0,0268	0,0016
10	50	0,0256	0,0254	0,0015
11	61	0,0294	0,034	0,0029
12	73	0,0515	0,0461	0,0002
13	86	0,0592	0,0599	0,0019
14	100	0,051	0,0266	0,0002
15	115	0,0611	0,0128	0,0021
16	131	0,0706	0,0192	0,0027
17	148	0,065	0,0188	0,0029
18	166	0,094	0,0152	0,0034
19	185	0,0851	0,0166	0,0036
20	205	0,0959	0,0317	0,0039

Tabla4. Comparativa de resultados de Backtracking con poda entre Lenguajes más grande.

La tabla permite general el siguiente gráfico que muestra de mejor forma las diferencias entre lenguajes a la hora de escalar el problema. En la Figura 3.

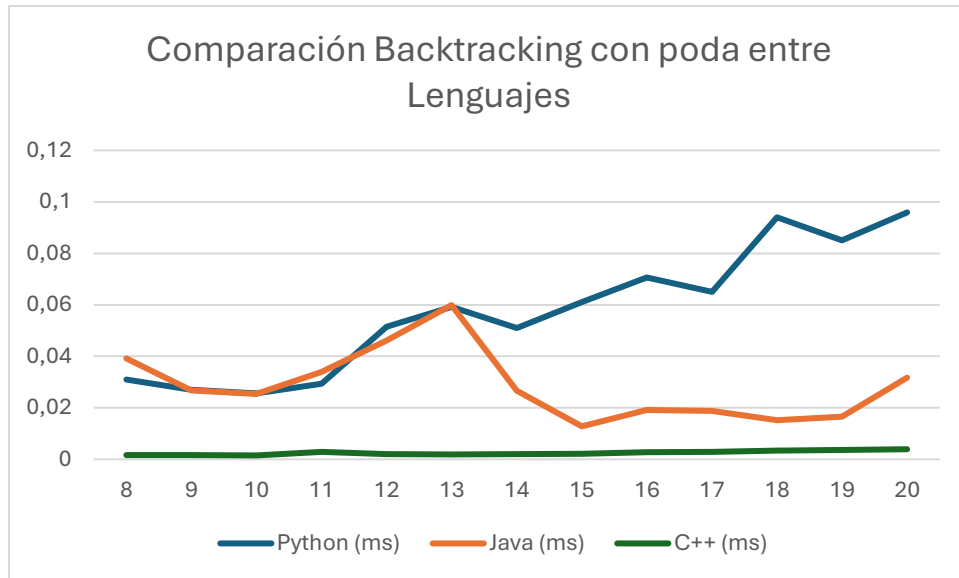


Figura 3. Tiempo de ejecución del backtracking con poda por lenguaje. Escala lineal.

Análisis

C++ es el más rápido en backtracking con poda: entre 15 y 24 más rápido que Java, y entre 10 y 22 más rápido que Python. El backtracking con poda es recursivo, es decir, cada expansión, significa una llamada a función. En C++ compilado con -O2, el compilador optimiza la recursión mediante inlining y optimización de registros. Por otro lado, Java usa ArrayList<Integer> con overhead de objetos en el heap y, si bien, es cierto que el JIT tarda en calentarse, a partir de $n = 14$, los tiempos bajan y se estabilizan, mejorando el rendimiento. En último lugar, Python tiene un overhead similar al de Java por llamada recursiva, pero sin el JIT.

7.3 Fuerza Bruta vs Backtracking con poda

La comparación directa entre ambas implementaciones entrego los siguientes resultados que, incluyen tanto una tabla como una figura respectiva para cada lenguaje:

Python:

n	Fuerza Bruta (ms)	Backtracking (ms)
8	2,3936	0,0192
9	16,6482	0,0228

10	149,5587	0,0299
11	1.482,49	0,0323
12	16.574,04	0,0392
13	206.264,10	0,0464

Tabla5. Comparativa de resultados de Python.

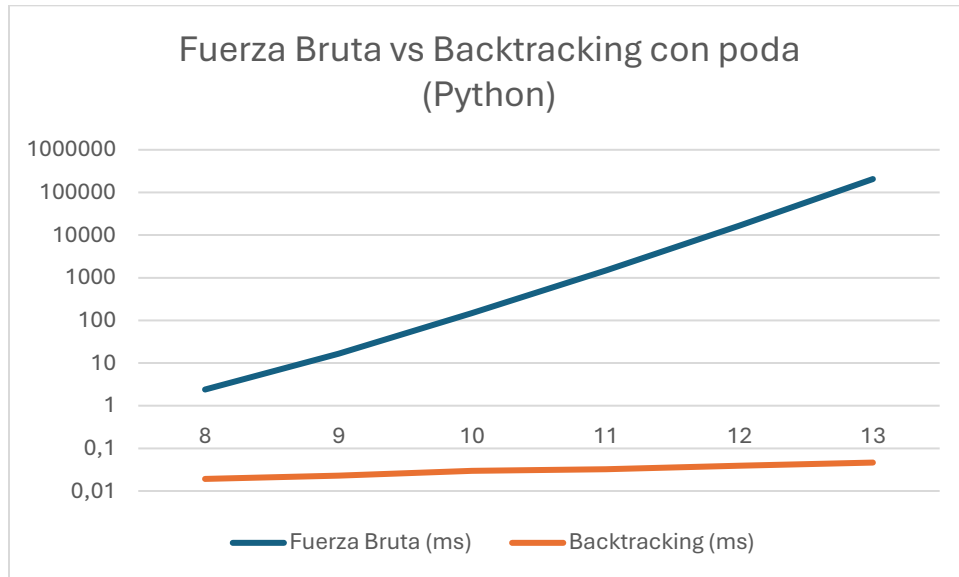


Figura 4. Fuerza Bruta vs Backtracking con poda en Python.

Java:

n	Fuerza Bruta (ms)	Backtracking (ms)
8	0,9469	0,0286
9	1,7783	0,0212
10	15,7758	0,0237
11	49,4242	0,0345
12	366,4325	0,0406
13	4.140,65	0,0508

Tabla6. Comparativa de resultados de Java.

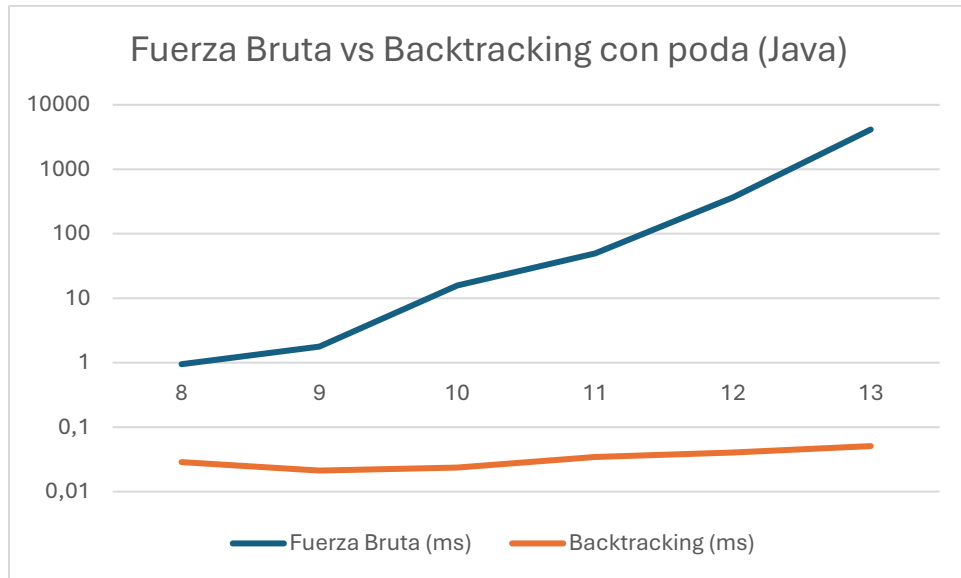


Figura 5. Fuerza Bruta vs Backtracking con poda en Java.

C++:

n	Fuerza Bruta (ms)	Backtracking (ms)
8	0,2816	0,0019
9	2,1335	0,0016
10	17,691	0,0016
11	184,0204	0,0019
12	1.901,29	0,0022
13	22.653,93	0,0021

Tabla7. Comparativa de resultados de c++.

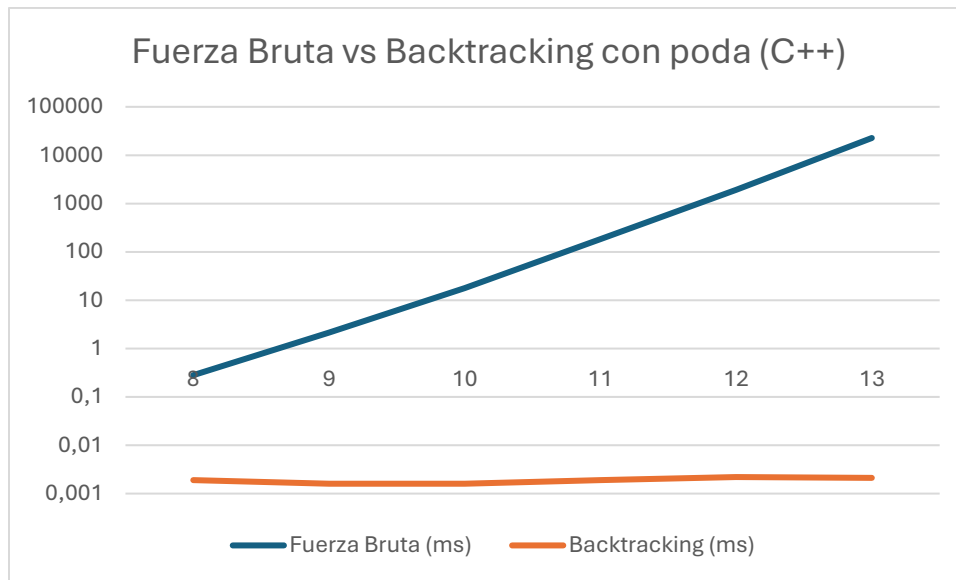


Figura 6. Fuerza Bruta vs Backtracking con poda en C++.

Análisis

En todos los lenguajes se puede ver crecimiento factorial de la fuerza bruta. Por otro lado. El backtracking con poda crece de forma casi constante. La velocidad de C++ en backtracking con poda es la más alta porque gestiona las llamadas recursivas de forma más directa y consumiendo menos memoria al ser un lenguaje compilado. Java no es tan bueno en backtracking con poda porque el tiempo que tarda en levantar la máquina virtual con cada llamada recursiva, lo ralentiza un poco, a pesar de eso, en fuerza bruta logro alcanzar la más alta velocidad gracias al compilador JIT. Por último, tenemos a Python que, al ser un lenguaje interpretado, tiene que identificar las variables, lo cual, si bien lo hace más flexible, en un problema con complejidad factorial, ese proceso tarda mucho más.

Limitaciones: Las implementaciones son exactas (no heurísticas), por lo que su uso práctico se limita a instancias pequeñas, ya que, como se pudo observar a lo largo del experimento de un n a otro n , se multiplican exponencialmente los tiempos de respuesta.

Alternativas: Implementar la heurística de Held-Karp, que, en lugar de recalcular las rutas, guarde los resultados de las sub-rutas ya visitadas en tablas (memoización). Por otro lado, tenemos alternativas como los SAT solvers, que

traducirían el grafo a fórmulas lógicas que se puede resolver en segundos.

8. Conclusiones

Los ciclos hamiltonianos dirigidos son problemas de decisión NP-Completo que muestra como escala la dificultad computacional en grafos grandes, mostrando la importancia de buscar nuevas alternativas que puedan optimizar los procesos en diferentes lenguajes. La fuerza bruta alcanza una barrera computacional a partir de $n=14$, tardando horas en Python. El backtracking con poda, en cambio, resuelve hasta $n=20$ en menos de 0,1 ms en todos los lenguajes. Finalmente, para escalar a grafos más grandes y complejos, sería mejor implementar la heurística de Held-Karp o cambiar hacia algoritmos de aproximación que sacrifiquen exactitud por velocidad.

9. Referencias

- [1] Karp, R. M. (1972). Reducibility among combinatorial problems. Complexity of Computer Computations, 85–103.
- [2] Garey, M. R., & Johnson, D. S. (1979). Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman.
- [3] Miller, C. E., Tucker, A. W., & Zemlin, R. A. (1960). Integer programming formulation of traveling salesman problems. Journal of the ACM, 7(4), 326–329. [3]
- [4] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.